BITS Pilani - Work Integrated Learning Programme (WILP)

Master of Technology (M.Tech)

Subject Name: Embedded System Design
Course Assignment: PVT Sensor Calibration System
Student Name: Manas Swain
Date: September 6, 2026
GitHub Repository: https://github.com/Manas-create/STM32-PVT-Sensor-Calibration

Mentor Approval
By signing below, I verify that the design and demonstration presented in this document have been vetted and validated.

Mentor Signature: _____ Date: _____

1. Problem Identification

In Digital VLSI Post Silicon Validation precise hardware simulation is required to calibrate on-chip PVT(Process, Voltage, and Temperature) conditions. Here my objective is to design a microcontroller based validation system that generates synchronized test vectors.

2. System Design

The system is modeled using the Wokwi simulation environment, utilizing the STM32C031C6 Nucleo board as the primary hardware controller. To validate the communication architecture without physical silicon, a logic analyzer is integrated into the simulation to capture and verify the generated waveforms.

Hardware Pin Configuration:

- PA5 (D13): SPI Clock (CLK)
- PA6 (D12): SPI Master In, Slave Out (MISO) - Simulated Telemetry
- PA7 (D11): SPI Master Out, Slave In (MOSI)
- PA1 (A1): Pulse Width Modulation (PWM) Wrapper

3. Firmware Design

The firmware is written in C. To overcome simulation limitations regarding hardware timer interrupts in the Wokwi environment, a bit-banging methodology was implemented. The microcontroller manually toggles the GPIO Output Data Registers (ODR) with accurate microsecond delays. This sequential approach guarantees that the 16-bit payload and the surrounding PWM voltage framing are generated with accurate hardware timing for the logic analyzer to capture.

Additionally, I made a custom Verilog-style software monitor ($monitor) to continuously poll the GPIO states and provide a real-time, text-based logic trace over UART for secondary validation.

4. Demonstration and Outcomes

4.1 System Architecture

The following screenshot demonstrates the active simulation environment, showcasing the STM32 microcontroller wired to the logic analyzer and potentiometer.

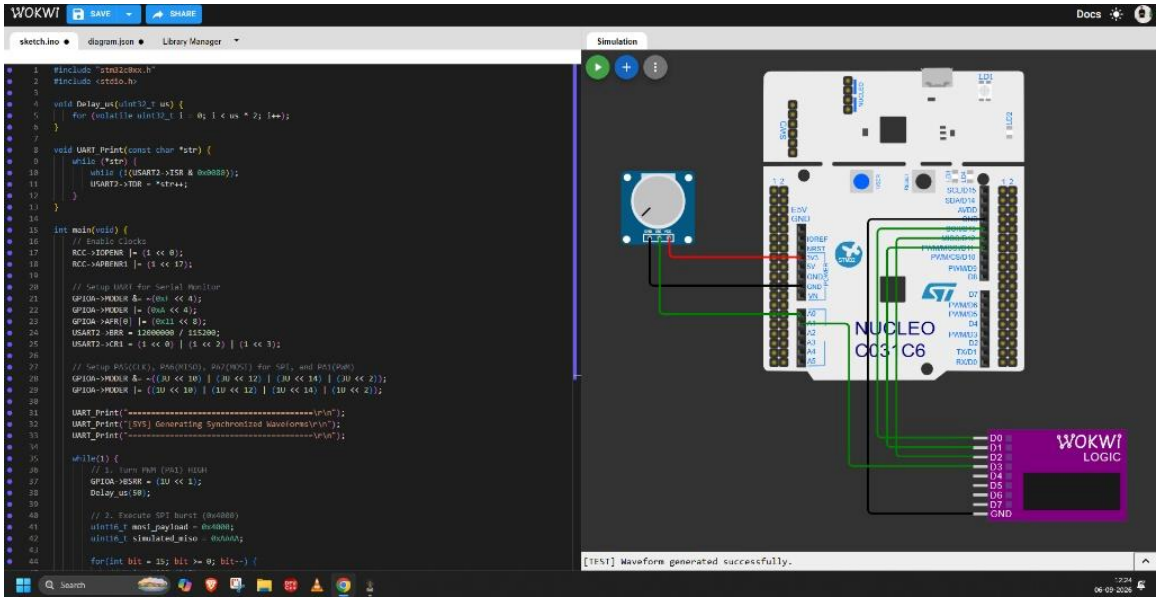

Figure 1: Wokwi simulation environment executing the primary waveform generation firmware.

4.2 Hardware Logic Trace

The captured PulseView waveform proves the successful generation of the required hardware signals. The transaction is properly bounded by the PWM wrapper (D3).

Exactly 16 clock pulses are generated (D0), and the MOSI line (D1) goes HIGH on the 14th bit, mathematically proving the transmission of the 0x4000 test payload.

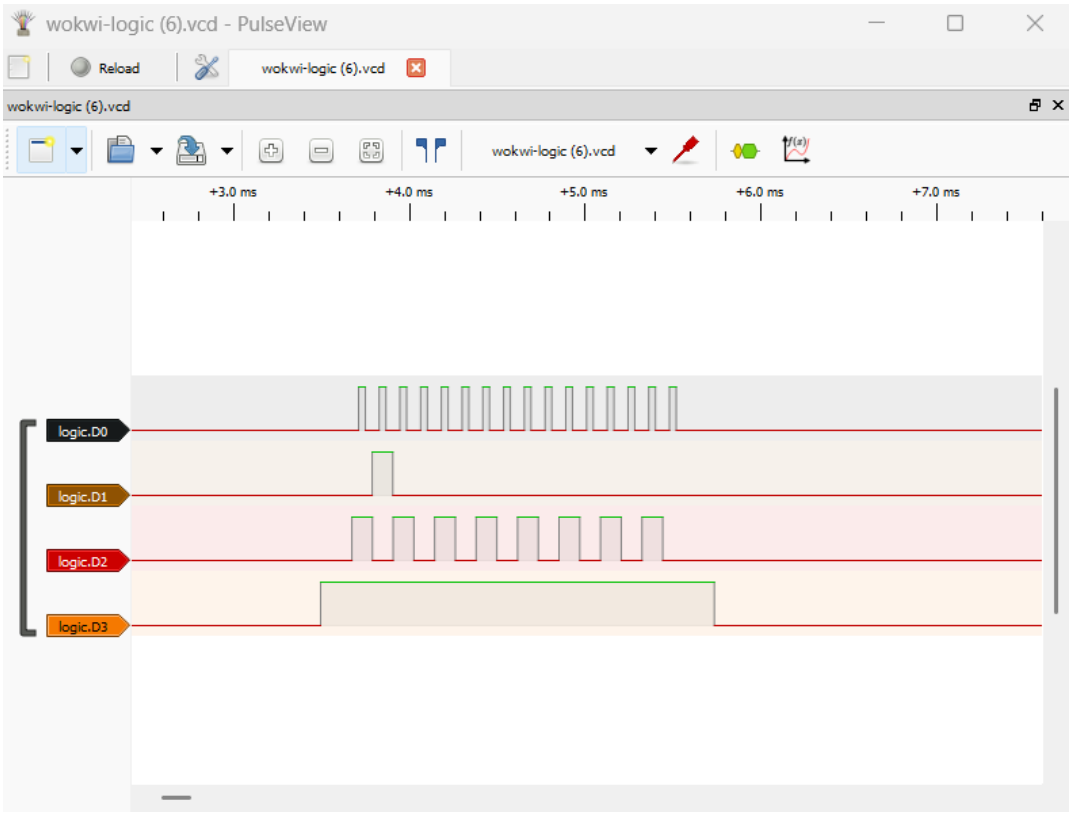

Figure 2: Logic analyzer trace validating the 16-bit SPI transaction (0x4000) on MOSI and simulated MISO telemetry.

4.3 Software Logic Trace (Serial Monitor)

To further validate the logic, a software-based monitor function was executed. The UART serial output below confirms the bit-by-bit state changes matching the hardware trace exactly.

```
--- INITIATING SPI TRANSACTION (0x4000) ---
[$monitor] CLK: 0 | MOSI: 0 | PWM: 1
[$monitor] CLK: 1 | MOSI: 0 | PWM: 1
[$monitor] CLK: 0 | MOSI: 0 | PWM: 1
[$monitor] CLK: 0 | MOSI: 1 | PWM: 1
[$monitor] CLK: 1 | MOSI: 1 | PWM: 1
[$monitor] CLK: 0 | MOSI: 1 | PWM: 1
[$monitor] CLK: 0 | MOSI: 0 | PWM: 1
[$monitor] CLK: 1 | MOSI: 0 | PWM: 1
[$monitor] CLK: 0 | MOSI: 0 | PWM: 1
[$monitor] CLK: 1 | MOSI: 0 | PWM: 1
[$monitor] CLK: 0 | MOSI: 0 | PWM: 1
[$monitor] CLK: 1 | MOSI: 0 | PWM: 1
[$monitor] CLK: 0 | MOSI: 0 | PWM: 1
[$monitor] CLK: 1 | MOSI: 0 | PWM: 1
[$monitor] CLK: 0 | MOSI: 0 | PWM: 1
```

Figure 3: Custom software-based $monitor serial trace verifying bit-level timing.

5. Appendix: Source Code

5.1 Primary Waveform Generation Code

```
#include "stm32c0xx.h"
#include <stdio.h>

void Delay_us(uint32_t us) {
    for (volatile uint32_t i = 0; i < us * 2; i++);
}

void UART_Print(const char *str) {
    while (*str) {
        while (!(USART2->ISR & 0x0080));
        USART2->TDR = *str++;
    }
}

int main(void) {
    // Enable Clocks
    RCC->IOPENR |= (1 << 0);
    RCC->APBENR1 |= (1 << 17);

    // Setup UART for Serial Monitor
    GPIOA->MODER &= ~(0xF << 4);
    GPIOA->MODER |= (0xA << 4);
    GPIOA->AFR[0] |= (0x11 << 8);
    USART2->BRR = 12000000 / 115200;
    USART2->CR1 = (1 << 0) | (1 << 2) | (1 << 3);

    // Setup PA5(CLK), PA6(MISO), PA7(MOSI) for SPI, and PA1(PWM)
    GPIOA->MODER &= ~((3U << 10) | (3U << 12) | (3U << 14) | (3U << 2));
    GPIOA->MODER |= ((1U << 10) | (1U << 12) | (1U << 14) | (1U << 2));

    UART_Print("========================================\r\n");
    UART_Print("[SYS] Generating Synchronized Waveforms\r\n");
    UART_Print("========================================\r\n");

    while(1) {
        // 1. Turn PWM (PA1) HIGH
        GPIOA->BSRR = (1U << 1);
        Delay_us(50);

        // 2. Execute SPI burst (0x4000)
        uint16_t mosi_payload = 0x4000;
        uint16_t simulated_miso = 0xAAAA;

        for(int bit = 15; bit >= 0; bit--) {
            // Write MOSI (PA7)
            if((mosi_payload >> bit) & 1) GPIOA->BSRR = (1U << 7);
            else                          GPIOA->BSRR = (1U << 23);

            // Force Simulated MISO (PA6)
            if((simulated_miso >> bit) & 1) GPIOA->BSRR = (1U << 6);
            else                            GPIOA->BSRR = (1U << 22);

            Delay_us(10);

            // Toggle Clock (PA5)
            GPIOA->BSRR = (1U << 5);  // HIGH
            Delay_us(10);
            GPIOA->BSRR = (1U << 21); // LOW
```

```
            Delay_us(10);
        }

        Delay_us(50);

        // 3. Turn PWM (PA1) LOW
        GPIOA->BSRR = (1U << 17);

        UART_Print("[TEST] Waveform generated successfully.\r\n");

        // Wait 1 second
        for(int d=0; d<100; d++) Delay_us(10000);
    }
}
```

5.2 Diagnostic $monitor Firmware

```
#include "stm32c0xx.h"
#include <stdio.h>

void Delay_us(uint32_t us) {
    for (volatile uint32_t i = 0; i < us * 2; i++);
}

void UART_Print(const char *str) {
    while (*str) {
        while (!(USART2->ISR & 0x0080));
        USART2->TDR = *str++;
    }
}

// Custom Verilog-style $monitor function
uint32_t prev_state = 0xFFFFFFFF;

void Verilog_Monitor(void) {
    uint32_t current_state = GPIOA->ODR;

    if (current_state != prev_state) {
        int clk  = (current_state >> 5) & 1;  // PA5
        int mosi = (current_state >> 7) & 1;  // PA7
        int pwm  = (current_state >> 1) & 1;  // PA1

        char mon_buf[60];
        sprintf(mon_buf, "[$monitor] CLK: %d | MOSI: %d | PWM: %d\r\n", clk,
mosi, pwm);
        UART_Print(mon_buf);

        prev_state = current_state;
    }
}

int main(void) {
    // Enable Clocks
    RCC->IOPENR |= (1 << 0);
    RCC->APBENR1 |= (1 << 17);

    // Setup UART
    GPIOA->MODER &= ~(0xF << 4);
    GPIOA->MODER |= (0xA << 4);
    GPIOA->AFR[0] |= (0x11 << 8);
    USART2->BRR = 12000000 / 115200;
    USART2->CR1 = (1 << 0) | (1 << 2) | (1 << 3);
```

```
    // Setup Pins
    GPIOA->MODER &= ~((3U << 10) | (3U << 12) | (3U << 14) | (3U << 2));
    GPIOA->MODER |= ((1U << 10) | (1U << 12) | (1U << 14) | (1U << 2));

    char uart_buf[100];

    while(1) {
        uint16_t duty = 500;
        UART_Print("\r\n--- INITIATING SPI TRANSACTION (0x4000) ---\r\n");

        GPIOA->BSRR = (1U << 1);
        Verilog_Monitor();
        Delay_us(10);

        for(int bit = 15; bit >= 0; bit--) {
            if((0x4000 >> bit) & 1) GPIOA->BSRR = (1U << 7);
            else                    GPIOA->BSRR = (1U << 23);
            Verilog_Monitor();

            Delay_us(10);
            GPIOA->BSRR = (1U << 5);
            Verilog_Monitor();
            Delay_us(10);
            GPIOA->BSRR = (1U << 21);
            Verilog_Monitor();
            Delay_us(10);
        }

        GPIOA->BSRR = (1U << 17);
        Verilog_Monitor();

        sprintf(uart_buf, "\r\n[TEST] PWM_REG: %d | V_ACT: 0.95V | SENS_RAW:
0x4000 | STATUS: OK\r\n", duty);
        UART_Print(uart_buf);

        for(int d=0; d<100; d++) Delay_us(10000);
    }
}
```